

System Design Case Study

Instagram Global System Collapse

Name: Ragini Singh

Cohort: Jensen Huang

Roll No: 150096724023

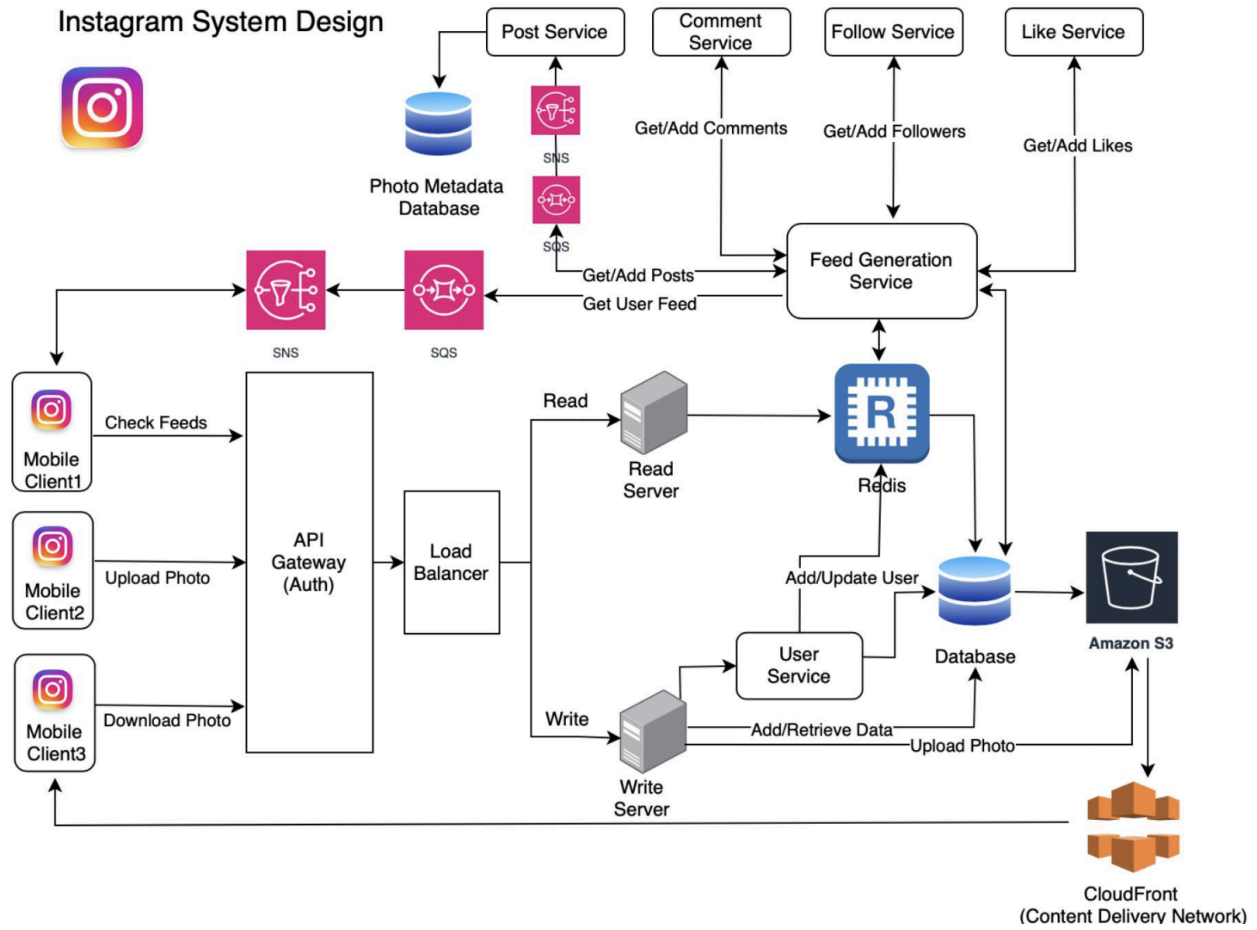
Date: 22 April 2026

SECTION A

Section A: System Architecture Failure Analysis

Q1. Instagram System Architecture & Failure Analysis

Instagram System Design



Explanation :

This diagram represents a large-scale Instagram-like system architecture.

Users interact with the system through mobile or web clients. Requests are handled by an API Gateway or Load Balancer, which distributes traffic across multiple microservices such as User Service, Post Service, Feed Service, Notification Service, and Ranking Service.

Different storage systems are used based on requirements: SQL databases for structured data, NoSQL for posts and media, Graph DB for user relationships, and sharded databases for scalability.

A cache layer is used to improve performance by storing frequently accessed data, while a CDN delivers media content globally with low latency.

An event streaming system is used for asynchronous processing such as notifications and feed updates. The system is deployed across multiple regions to serve global users efficiently.

End-to-End Feed Flow:

1. User opens the app and requests feed
2. Request goes to API Gateway
3. Gateway forwards request to Feed Service
4. Feed Service fetches data from cache or database
5. Ranking Service ranks posts using ML
6. Final feed is sent back to user

Failure Points :

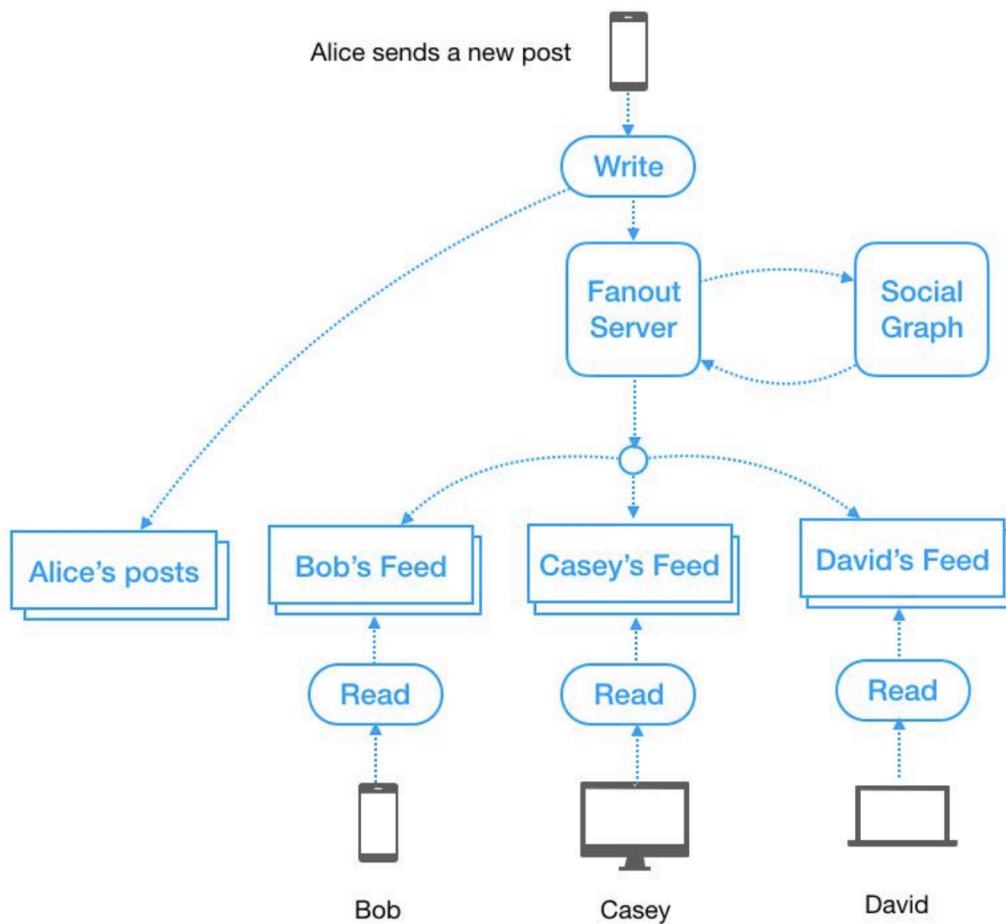
- Feed latency increases due to heavy load
- Infinite scroll fails due to slow feed generation
- Likes and comments become inconsistent
- Hot shard problem due to viral content
- Event queue backlog delays notifications
- Cache inconsistency across regions

Design Flaws:

- Over-reliance on real-time feed computation
- Static sharding leads to uneven load distribution
- Event queues are not scalable under peak traffic
- Weak cache invalidation strategy

Section B: Feed System Engineering at Scale

Q2: Why Instagram Feed Cannot Be Fully Real-Time



Explanation :

Instagram-style feed systems cannot rely completely on real-time computation because of massive scale and performance limitations.

In fan-out-on-write, when a user posts content, the system immediately pushes it to all followers' feeds. This works well for users with fewer followers but fails for celebrities with millions of followers due to high write load.

In fan-out-on-read, the feed is generated in real-time when a user opens the app. This causes high read latency and increases system load.

To solve this, modern systems use a hybrid model. Frequently accessed data is precomputed and stored in cache, while some parts of the feed are generated dynamically.

This approach balances performance, scalability, and freshness of data.

Comparison :

Fan-out-on-write:

- Fast read
- Heavy write load

Fan-out-on-read:

- Low write load
- Slow read performance

Hybrid model:

- Balanced approach
- Better scalability

Load Explosion:

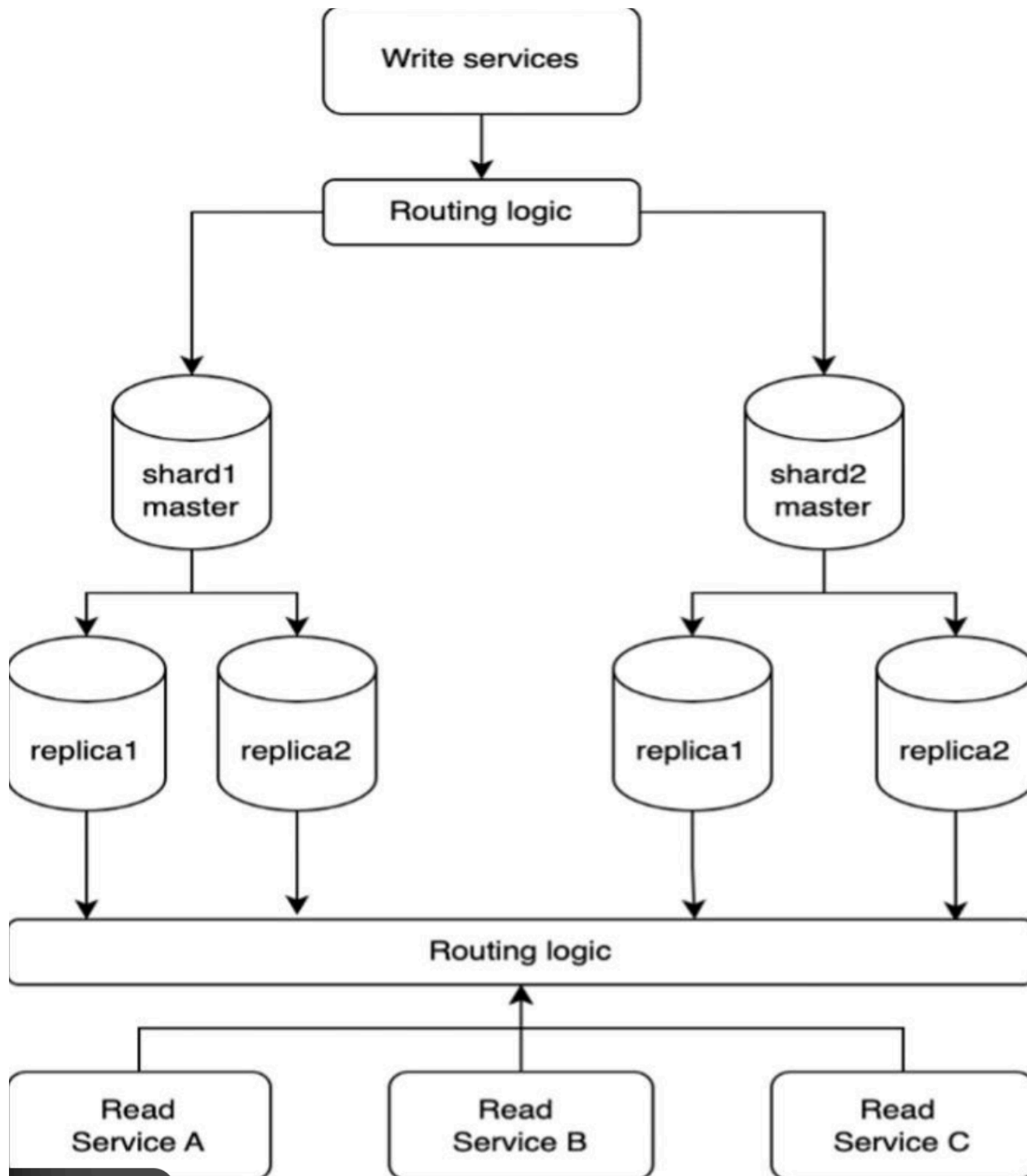
During viral events, fan-out-on-write causes massive load because content needs to be pushed to millions of users.

Fan-out-on-read also struggles because too many users request feed at the same time.

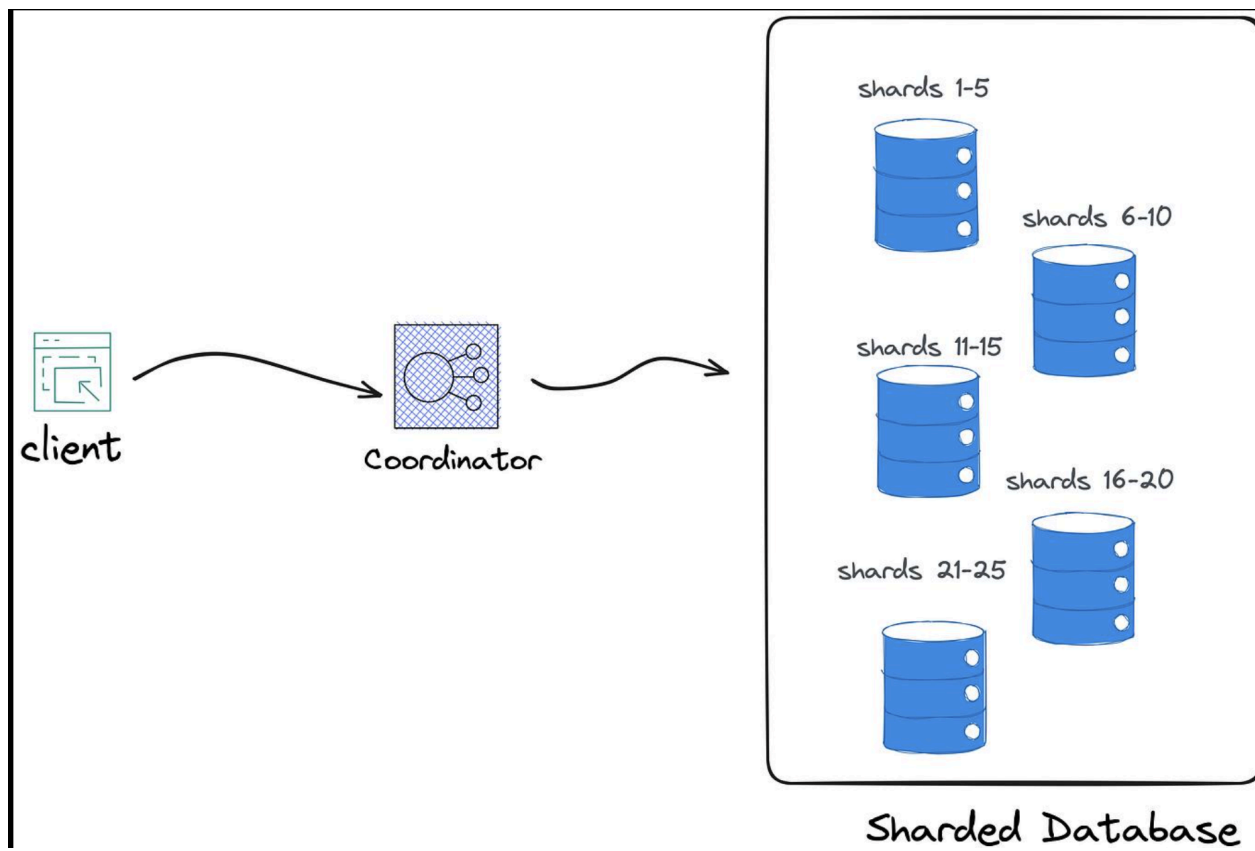
Hybrid systems reduce this load using caching and precomputation.

Section C: Sharding and Load Imbalance

Q3: Before Hot Shard Problem & Solution



AFTER (Solution System)



Explanation :

In social media systems, data is distributed across multiple database shards. However, static sharding can lead to uneven load distribution.

When content goes viral, a large number of requests are directed to a single shard, creating a hot shard problem. This results in slow performance and system failure.

To solve this, modern systems use dynamic sharding and load balancing. Data is distributed using techniques like consistent hashing, and traffic is balanced across multiple shards and regions.

This ensures even load distribution and better scalability.

Why Static Sharding Fails :

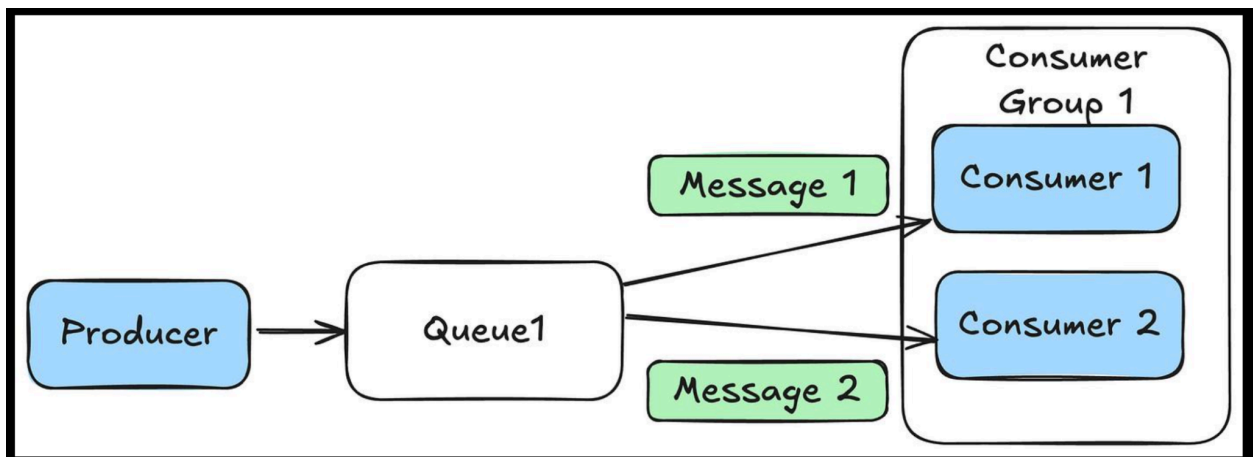
- Data is fixed to specific shards
- Cannot handle sudden traffic spikes
- Leads to uneven load distribution

Solution :

- Use dynamic sharding (consistent hashing)
- Add load balancer
- Use multi-region shard distribution
- Auto-scale shards during high traffic

Section D: Event Streaming System Failure

Q4: Notification Delay Problem & Solution



Explanation:

This diagram shows an improved event-driven architecture for handling notifications.

When a user performs an action such as posting or liking, the request goes to the API server and is pushed into a message queue.

The queue sends events to a Pub/Sub system, which distributes them to multiple services like notifications, SMS, and email.

During high traffic, the queue can become overloaded, causing delays. To handle failures, a Dead Letter Queue (DLQ) is used to store failed messages for retry.

Backpressure mechanisms are used to control the flow of data and prevent system overload.

Why Queues Alone Fail :

- Queue gets overloaded during peak traffic
- Messages are delayed
- System cannot process events fast enough

DLQ Role :

- Stores failed messages
- Prevents data loss
- Allows retry mechanism

Backpressure Role

- Controls incoming traffic
- Slows down producers when system is overloaded
- Prevents system crash

Section E: Cache and Consistency Breakdown

Q5: Cache Inconsistency in Distributed Systems

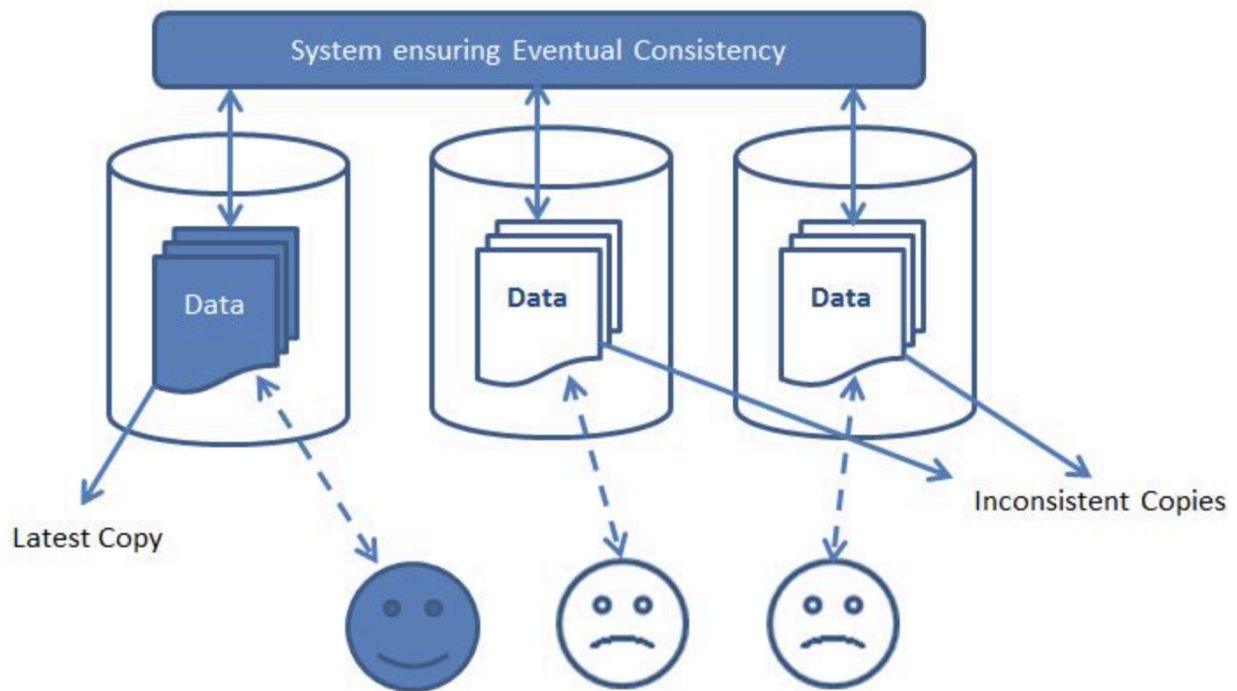


Figure showing a stale state where 2 copies of data are inconsistent with the latest one.

Explanation :

Cache inconsistency occurs when different regions have different versions of data.

In distributed systems, data updates do not reach all caches at the same time. As a result, users in different regions may see different data.

CDN helps in faster content delivery but does not guarantee consistency of dynamic data.

There is a trade-off between strong consistency and eventual consistency. Strong consistency ensures accurate data but is slower, while eventual consistency is faster but may show temporary differences.